

Exercises for
Concrete Semantics

Tobias Nipkow Gerwin Klein

January 20, 2026

This document collects together the exercises in the book *Concrete Semantics* in extended form. The exercises are described in more detail and with additional hints. Corresponding templates for solutions are available from the [home page of the book](#) in the form of theory files. Exercises that do not require Isabelle are omitted.

Chapter 2

Exercise 2.1. Use the `value` command to evaluate the following expressions:

```
"1 + (2::nat)" "1 + (2::int)" "1 - (2::nat)" "1 - (2::int)" "[a,b] @ [c,d]"
```

Exercise 2.2. Recall the definition of our own addition function on `nat`:

```
fun add :: "nat ⇒ nat ⇒ nat" where
  "add 0 n = n" |
  "add (Suc m) n = Suc(add m n)"
```

Prove that `add` is associative and commutative. You will need additional lemmas.

```
lemma add_assoc: "add (add m n) p = add m (add n p)"
lemma add_comm: "add m n = add n m"
```

Define a recursive function

```
fun double :: "nat ⇒ nat"
```

and prove that

```
lemma double_add: "double m = add m m"
```

Exercise 2.3. Define a function that counts the number of occurrences of an element in a list:

```
fun count :: "'a list ⇒ 'a ⇒ nat"
```

Test your definition of `count` on some examples. Prove the following inequality:

```
theorem "count xs x ≤ length xs"
```

Exercise 2.4. Define a function `snoc` that appends an element to the end of a list. Do not use the existing append operator `@` for lists.

```
fun snoc :: "'a list ⇒ 'a ⇒ 'a list"
```

Convince yourself on some test cases that your definition of `snoc` behaves as expected. With the help of `snoc` define a recursive function `reverse` that reverses a list. Do not use the predefined function `rev`.

```
fun reverse :: "'a list ⇒ 'a list"
```

Prove the following theorem. You will need an additional lemma.

```
theorem "reverse (reverse xs) = xs"
```

Exercise 2.5. The aim of this exercise is to prove the summation formula

$$\sum_{i=0}^n i = \frac{n(n+1)}{2}$$

Define a recursive function `sum_upto` $n = 0 + \dots + n$:

```
fun sum_upto :: "nat  $\Rightarrow$  nat"
```

Now prove the summation formula by induction on n . First, write a clear but informal proof by hand following the examples in the main text. Then prove the same property in Isabelle:

```
lemma "sum_upto n = n * (n+1) div 2"
```

Exercise 2.6. Starting from the type $'a$ tree defined in the text, define a function that collects all values in a tree in a list, in any order, without removing duplicates.

```
datatype 'a tree = Tip | Node "'a tree" 'a "'a tree"
```

```
fun contents :: "'a tree  $\Rightarrow$  'a list"
```

Then define a function that sums up all values in a tree of natural numbers

```
fun sum_tree :: "nat tree  $\Rightarrow$  nat"
```

and prove

```
lemma "sum_tree t = sum_list(contents t)"
```

Exercise 2.7. Define a new type $'a$ tree2 of binary trees where values are also stored in the leaves of the tree. Also reformulate the *mirror* function accordingly. Define two functions

```
fun pre_order :: "'a tree2  $\Rightarrow$  'a list"
```

```
fun post_order :: "'a tree2  $\Rightarrow$  'a list"
```

that traverse a tree and collect all stored values in the respective order in a list. Prove

```
lemma "pre_order (mirror t) = rev (post_order t)"
```

Exercise 2.8. Define a recursive function

```
fun intersperse :: "'a  $\Rightarrow$  'a list  $\Rightarrow$  'a list"
```

such that $\text{intersperse } a [x_1, \dots, x_n] = [x_1, a, x_2, a, \dots, a, x_n]$. Prove

```
lemma "map f (intersperse a xs) = intersperse (f a) (map f xs)"
```

Exercise 2.9. Write a tail-recursive variant of the *add* function on *nat*:

```
fun itadd :: "nat  $\Rightarrow$  nat  $\Rightarrow$  nat"
```

Tail-recursive means that in the recursive case, *itadd* needs to call itself directly: $\text{itadd } (\text{Suc } m) n = \text{itadd } \dots$. Prove

```
lemma "itadd m n = add m n"
```

Exercise 2.10. Define a datatype *tree0* of binary tree skeletons which do not store any information, neither in the inner nodes nor in the leaves. Define a function that counts the number of all nodes (inner nodes and leaves) in such a tree:

```
fun nodes :: "tree0  $\Rightarrow$  nat"
```

Consider the following recursive function:

```
fun explode :: "nat  $\Rightarrow$  tree0  $\Rightarrow$  tree0" where
  "explode 0 t = t" |
  "explode (Suc n) t = explode n (Node t t)"
```

Experiment how *explode* influences the size of a binary tree and find an equation expressing the size of a tree after exploding it (*nodes (explode n t)*) as a function of *nodes t* and *n*. Prove your equation. You may use the usual arithmetic operations including the exponentiation operator $\wedge$. For example, $2 \wedge 2 = 4$.

Hint: simplifying with the list of theorems *algebra_simps* takes care of common algebraic properties of the arithmetic operators.

Exercise 2.11. Define arithmetic expressions in one variable over integers (type *int*) as a data type:

```
datatype exp = Var | Const int | Add exp exp | Mult exp exp
```

Define a function *eval* that evaluates an expression at some value:

```
fun eval :: "exp  $\Rightarrow$  int  $\Rightarrow$  int"
```

For example, *eval (Add (Mult (Const 2) Var) (Const 3)) i* = $2 * i + 3$.

A polynomial can be represented as a list of coefficients, starting with the constant. For example, $[4, 2, -1, 3]$ represents the polynomial $4 + 2x - x^2 + 3x^3$. Define a function *evalp* that evaluates a polynomial at a given value:

```
fun evalp :: "int list  $\Rightarrow$  int  $\Rightarrow$  int"
```

Define a function *coeffs* that transforms an expression into a polynomial. This will require auxiliary functions.

```
fun coeffs :: "exp  $\Rightarrow$  int list"
```

Prove that *coeffs* preserves the value of the expression:

```
theorem evalp_coeffs: "evalp (coeffs e) x = eval e x"
```

Hint: consider the hint in Exercise 2.10.

Chapter 3

Exercise 3.1. To show that *asimp_const* really folds all subexpressions of the form *Plus (N i) (N j)*, define a function

```
fun optimal :: "aexp ⇒ bool"
```

that checks that its argument does not contain a subexpression of the form *Plus (N i) (N j)*. Then prove that the result of *asimp_const* is optimal:

```
lemma "optimal (asimp_const a)"
```

This proof needs the same *split*: directive as the correctness proof of *asimp_const*. This increases the chance of nontermination of the simplifier. Therefore *optimal* should be defined purely by pattern matching on the left-hand side, without *case* expressions on the right-hand side.

Exercise 3.2. In this exercise we verify constant folding for *aexp* where we sum up all constants, even if they are not next to each other. For example, *Plus (N 1) (Plus (V x) (N 2))* becomes *Plus (V x) (N 3)*. This goes beyond *asimp*. Below we follow a particular solution strategy but there are many others.

First, define a function *sumN* that returns the sum of all constants in an expression and a function *zeroN* that replaces all constants in an expression by zeroes (they will be optimized away later):

```
fun sumN :: "aexp ⇒ int"
```

```
fun zeroN :: "aexp ⇒ aexp"
```

Next, define a function *sepN* that produces an arithmetic expression that adds the results of *sumN* and *zeroN*. Prove that *sepN* preserves the value of an expression.

```
definition sepN :: "aexp ⇒ aexp"
```

```
lemma aval_sepN: "aval (sepN t) s = aval t s"
```

Finally, define a function *full_asimp* that uses *asimp* to eliminate the zeroes left over by *sepN*. Prove that it preserves the value of an arithmetic expression.

```
definition full_asimp :: "aexp ⇒ aexp"
```

```
lemma aval_full_asimp: "aval (full_asimp t) s = aval t s"
```

Exercise 3.3. Substitution is the process of replacing a variable by an expression in an expression. Define a substitution function

```
fun subst :: "vname ⇒ aexp ⇒ aexp ⇒ aexp"
```

such that *subst x a e* is the result of replacing every occurrence of variable *x* by *a* in *e*. For example:

```
subst "x" (N 3) (Plus (V "x") (V "y")) = Plus (N 3) (V "y")
```

Prove the so-called **substitution lemma** that says that we can either substitute first and evaluate afterwards or evaluate with an updated state:

lemma *subst_lemma*: "aval (subst x a e) s = aval e (s(x := aval a s))"

As a consequence prove that we can substitute equal expressions by equal expressions and obtain the same result under evaluation:

lemma "aval a1 s = aval a2 s
 $\implies$ aval (subst x a1 e) s = aval (subst x a2 e) s"

Exercise 3.4. Take a copy of theory *AExp* and modify it as follows. Extend type *aexp* with a binary constructor *Times* that represents multiplication. Modify the definition of the functions *aval* and *asimp* accordingly. You can remove *asimp_const*. Function *asimp* should eliminate 0 and 1 from multiplications as well as evaluate constant subterms. Update all proofs concerned.

Exercise 3.5. Define a datatype *aexp2* of extended arithmetic expressions that has, in addition to the constructors of *aexp*, a constructor for modelling a C-like post-increment operation $x++$, where x must be a variable. Define an evaluation function *aval2* :: *aexp2* $\Rightarrow$ *state* $\Rightarrow$ *val* $\times$ *state* that returns both the value of the expression and the new state. The latter is required because post-increment changes the state.

Extend *aexp2* and *aval2* with a division operation. Model partiality of division by changing the return type of *aval2* to (*val* $\times$ *state*) *option*. In case of division by 0 let *aval2* return *None*. Division on *int* is the infix *div*.

Exercise 3.6. The following type adds a *LET* construct to arithmetic expressions:

datatype *lexp* = *Nl* int | *Vl* *vname* | *Plusl* *lexp* *lexp* | *LET* *vname* *lexp* *lexp*

The *LET* constructor introduces a local variable: the value of *LET* $x\ e_1\ e_2$ is the value of e_2 in the state where x is bound to the value of e_1 in the original state. Define a function *bval* :: *lexp* $\Rightarrow$ *state* $\Rightarrow$ *int* that evaluates *lexp* expressions. Remember $s(x := i)$.

Define a conversion *inline* :: *lexp* $\Rightarrow$ *aexp*. The expression *LET* $x\ e_1\ e_2$ is inlined by substituting the converted form of e_1 for x in the converted form of e_2 . See Exercise 3.3 for more on substitution. Prove that *inline* is correct w.r.t. evaluation.

Exercise 3.7. Show that equality and less-or-equal tests on *aexp* are definable

definition *Le* :: "aexp $\Rightarrow$ aexp $\Rightarrow$ bexp"

definition *Eq* :: "aexp $\Rightarrow$ aexp $\Rightarrow$ bexp"

and prove that they do what they are supposed to:

lemma *bval_Le*: "bval (Le a1 a2) s = (aval a1 s $\leq$ aval a2 s)"

lemma *bval_Eq*: "bval (Eq a1 a2) s = (aval a1 s = aval a2 s)"

Exercise 3.8. Consider an alternative type of boolean expressions featuring a conditional:

```
datatype ifexp = Bc2 bool | If ifexp ifexp ifexp | Less2 aexp aexp
```

First define an evaluation function analogously to *bval*:

```
fun ifval :: "ifexp  $\Rightarrow$  state  $\Rightarrow$  bool"
```

Then define two translation functions

```
fun b2ifexp :: "bexp  $\Rightarrow$  ifexp"
```

```
fun if2bexp :: "ifexp  $\Rightarrow$  bexp"
```

and prove their correctness:

```
lemma "bval (if2bexp exp) s = ifval exp s"
```

```
lemma "ifval (b2ifexp exp) s = bval exp s"
```

Exercise 3.9. We define a new type of purely boolean expressions without any arithmetic

```
datatype pbexp =
```

```
  VAR vname | NOT pbexp | AND pbexp pbexp | OR pbexp pbexp
```

where variables range over values of type *bool*, as can be seen from the evaluation function:

```
fun pbval :: "pbexp  $\Rightarrow$  (vname  $\Rightarrow$  bool)  $\Rightarrow$  bool" where
```

```
"pbval (VAR x) s = s x" |
```

```
"pbval (NOT b) s = ( $\neg$  pbval b s)" |
```

```
"pbval (AND b1 b2) s = (pbval b1 s  $\wedge$  pbval b2 s)" |
```

```
"pbval (OR b1 b2) s = (pbval b1 s  $\vee$  pbval b2 s)"
```

Define a function that checks whether a boolean expression is in NNF (negation normal form), i.e., if *NOT* is only applied directly to *VAR*s:

```
fun is_nnf :: "pbexp  $\Rightarrow$  bool"
```

Now define a function that converts a *bexp* into NNF by pushing *NOT* inwards as much as possible:

```
fun nnf :: "pbexp  $\Rightarrow$  pbexp"
```

Prove that *nnf* does what it is supposed to do:

```
lemma pbval_nnf: "pbval (nnf b) s = pbval b s"
```

```
lemma is_nnf_nnf: "is_nnf (nnf b)"
```

An expression is in DNF (disjunctive normal form) if it is in NNF and if no *OR* occurs below an *AND*. Define a corresponding test:

```
fun is_dnf :: "pbexp  $\Rightarrow$  bool"
```

An NNF can be converted into a DNF in a bottom-up manner. The critical case is the conversion of *AND* *b*₁ *b*₂. Having converted *b*₁ and *b*₂, apply distributivity of *AND* over *OR*. If we write *OR* as a multi-argument function, we can express the distributivity step as follows: *dist_AND* (*OR* *a*₁ ... *a*_{*n*}) (*OR* *b*₁ ... *b*_{*m*}) = *OR* (*AND* *a*₁ *b*₁) (*AND* *a*₁ *b*₂) ... (*AND* *a*_{*n*} *b*_{*m*}). Define

```
fun dist_AND :: "pexp  $\Rightarrow$  pexp  $\Rightarrow$  pexp"
```

and prove that it behaves as follows:

```
lemma pbval_dist: "pbval (dist_AND b1 b2) s = pbval (AND b1 b2) s"
```

```
lemma is_dnf_dist: "is_dnf b1  $\implies$  is_dnf b2  $\implies$  is_dnf (dist_AND b1 b2)"
```

Use *dist_AND* to write a function that converts an NNF to a DNF in the above bottom-up manner.

```
fun dnf_of_nnf :: "pexp  $\Rightarrow$  pexp"
```

Prove the correctness of your function:

```
lemma "pbval (dnf_of_nnf b) s = pbval b s"
```

```
lemma "is_nnf b  $\implies$  is_dnf (dnf_of_nnf b)"
```

Exercise 3.10. A **stack underflow** occurs when executing an *ADD* instruction on a stack of size less than 2. In our semantics a term *exec1 ADD s stk* where *length stk < 2* is simply some unspecified value, not an error or exception — HOL does not have those concepts. Modify theory *ASM* such that stack underflow is modelled by *None* and normal execution by *Some*, i.e., the execution functions have return type *stack option*. Modify all theorems and proofs accordingly. Hint: you may find *split: option.split* useful in your proofs.

Exercise 3.11. This exercise is about a register machine and compiler for *aexp*. The machine instructions are

```
datatype instr = LDI val reg | LD vname reg | ADD reg reg
```

where type *reg* is a synonym for *nat*. Instruction *LDI i r* loads *i* into register *r*, *LD x r* loads the value of *x* into register *r*, and *ADD r₁ r₂* adds register *r₂* to register *r₁*.

Define the execution of an instruction given a state and a register state; the result is the new register state:

```
type_synonym rstate = "reg  $\Rightarrow$  val"
```

```
fun exec1 :: "instr  $\Rightarrow$  state  $\Rightarrow$  rstate  $\Rightarrow$  rstate"
```

Define the execution *exec* of a list of instructions as for the stack machine.

The compiler takes an arithmetic expression *a* and a register *r* and produces a list of instructions whose execution places the value of *a* into *r*. The registers *> r* should be used in a stack-like fashion for intermediate results, the ones *< r* should be left alone. Define the compiler and prove it correct:

```
theorem "exec (comp a r) s rs r = aval a s"
```

Exercise 3.12. This exercise is a variation of the previous one with a different instruction set:

```
datatype instr0 = LDI0 val | LD0 vname | MV0 reg | ADD0 reg
```

All instructions refer implicitly to register 0 as a source or target: *LDI0* and *LD0* load a value into register 0, *MV0 r* copies the value in register 0 into register *r*, and *ADD0 r* adds the value in register *r* to the value in register 0; *MV0 0* and *ADD0 0* are legal. Define the execution functions

`fun exec01 :: "instr0  $\Rightarrow$  state  $\Rightarrow$  rstate  $\Rightarrow$  rstate"`

and *exec0* for instruction lists.

The compiler takes an arithmetic expression *a* and a register *r* and produces a list of instructions whose execution places the value of *a* into register 0. The registers $> r$ should be used in a stack-like fashion for intermediate results, the ones $\leq r$ should be left alone (with the exception of 0). Define the compiler and prove it correct:

`theorem "exec0 (comp0 a r) s rs 0 = aval a s"`

Chapter 4

Exercise 4.1. Start from the data type of binary trees defined earlier:

```
datatype 'a tree = Tip | Node "'a tree" 'a "'a tree"
```

An *int tree* is ordered if for every *Node l i r* in the tree, *l* and *r* are ordered and all values in *l* are $< i$ and all values in *r* are $> i$. Define a function that returns the elements in a tree and one the tests if a tree is ordered:

```
fun set :: "'a tree  $\Rightarrow$  'a set"
fun ord :: "int tree  $\Rightarrow$  bool"
```

Hint: use quantifiers.

Define a function *ins* that inserts an element into an ordered *int tree* while maintaining the order of the tree. If the element is already in the tree, the same tree should be returned.

```
fun ins :: "int  $\Rightarrow$  int tree  $\Rightarrow$  int tree"
```

Prove correctness of *ins*:

```
lemma set_ins: "set(ins x t) = {x}  $\cup$  set t"
theorem ord_ins: "ord t  $\implies$  ord(ins i t)"
```

Exercise 4.2. Formalize the following definition of palindromes

- The empty list and a singleton list are palindromes.
- If *xs* is a palindrome, so is $a \# xs @ [a]$.

as an inductive predicate

```
inductive palindrome :: "'a list  $\Rightarrow$  bool"
```

and prove

```
lemma "palindrome xs  $\implies$  rev xs = xs"
```

Exercise 4.3. We could also have defined *star* as follows:

```
inductive star' :: "('a  $\Rightarrow$  'a  $\Rightarrow$  bool)  $\Rightarrow$  'a  $\Rightarrow$  'a  $\Rightarrow$  bool" for r where
  refl': "star' r x x" |
  step': "star' r x y  $\implies$  r y z  $\implies$  star' r x z"
```

The single *r* step is performed after rather than before the *star'* steps. Prove

```
lemma "star' r x y  $\implies$  star r x y"
lemma "star r x y  $\implies$  star' r x y"
```

You may need lemmas. Note that rule induction fails if the assumption about the inductive predicate is not the first assumption.

Exercise 4.4. Analogous to *star*, give an inductive definition of the *n*-fold iteration of a relation *r*: *iter r n x y* should hold if there are $x_0, \dots, x_n$ such that $x = x_0$, $x_n = y$ and $r x_i x_{i+1}$ for all $i < n$:

inductive *iter* :: "(*a* ⇒ *a* ⇒ bool) ⇒ nat ⇒ *a* ⇒ *a* ⇒ bool" for *r* where

Correct and prove the following claim:

lemma "star *r* *x* *y* ⇒ iter *r* *n* *x* *y*"

Exercise 4.5. A context-free grammar can be seen as an inductive definition where each nonterminal *A* is an inductively defined predicate on lists of terminal symbols: *A*(*w*) means that *w* is in the language generated by *A*. For example, the production *S* → *aSb* can be viewed as the implication *S w* ⇒ *S (a # w @ [b])* where *a* and *b* are terminal symbols, i.e., elements of some alphabet. The alphabet can be defined as a datatype:

datatype *alpha* = *a* | *b*

If you think of *a* and *b* as "(" and ")", the following two grammars both generate strings of balanced parentheses (where ε is the empty word):

$$\begin{array}{lcl} S & \rightarrow & \varepsilon \quad | \quad aSb \quad | \quad SS \\ T & \rightarrow & \varepsilon \quad | \quad TaTb \end{array}$$

Define them as inductive predicates and prove their equivalence:

inductive *S* :: "alpha list ⇒ bool"

inductive *T* :: "alpha list ⇒ bool"

lemma *TS*: "*T w* ⇒ *S w*"

lemma *ST*: "*S w* ⇒ *T w*"

corollary *SeqT*: "*S w* ⇔ *T w*"

Exercise 4.6. In Chapter 3 we defined a recursive evaluation function *aval* :: *aexp* ⇒ *state* ⇒ *val*. Define an inductive evaluation predicate and prove that it agrees with the recursive function:

inductive *aval_rel* :: "aexp ⇒ state ⇒ val ⇒ bool"

lemma *aval_rel_aval*: "*aval_rel a s v* ⇒ *aval a s = v*"

lemma *aval_aval_rel*: "*aval a s = v* ⇒ *aval_rel a s v*"

corollary "*aval_rel a s v* ⇔ *aval a s = v*"

Exercise 4.7. Consider the stack machine from Chapter 3 and recall the concept of **stack underflow** from Exercise 3.10. Define an inductive predicate

inductive *ok* :: "nat ⇒ instr list ⇒ nat ⇒ bool"

such that *ok n is n'* means that with any initial stack of length *n* the instructions *is* can be executed without stack underflow and that the final stack has length *n'*.

Using the introduction rules for *ok*, prove the following special cases:

lemma "*ok* 0 [*LOAD x*] (*Suc* 0)"

lemma "*ok* 0 [*LOAD x*, *LOADI v*, *ADD*] (*Suc* 0)"

lemma "*ok* (*Suc* (*Suc* 0)) [*LOAD x*, *ADD*, *ADD*, *LOAD y*] (*Suc* (*Suc* 0))"

Prove that *ok* correctly computes the final stack size:

lemma " $\llbracket ok\ n\ is\ n';\ length\ stk = n \rrbracket \implies length\ (exec\ is\ s\ stk) = n'$ "

Lemma *length_Suc_conv* may come in handy.

Prove that instruction sequences generated by *comp* cannot cause stack underflow: *ok n (comp a) ?* for some suitable value of *?*.